

程序设计方法与艺术

解 题 报 告

班
组
组

级：软件工程 25-9 班
长：杜雨龙 2025218896
员：李泽浩 2025218895
田浩然 2025218885
李铭毅 2025218921

题目 A 间距调整\Brake distance

在智能网联汽车队列中，车辆通过 V2V 通信协调行驶，并实时调整与前车的车间距，定义当前车间距为一个正整数，初始间距为 1（单位：米）。一种自适应间距控制策略允许在每一轮控制周中，对当前间距执行三种原子操作之一：

- A 操作（线性增加）：令间距 $+1$ 。
- B 操作（倍增扩大）：令间距 $\times 2$ 。
- C 操作（回退缩小）：令间距 -1 ，但仅当当前间距严格大于 1 时允许执行使间距始终保持为正整数。

每执行一次操作计为一步。现给定目标间距 n ，请你构造一条从 1 到 n 的操作序列，使总步数最少。若存在多条最少步数的序列，输出字典序最小的操作串。

时间限制： 1.0 秒

解题思路：

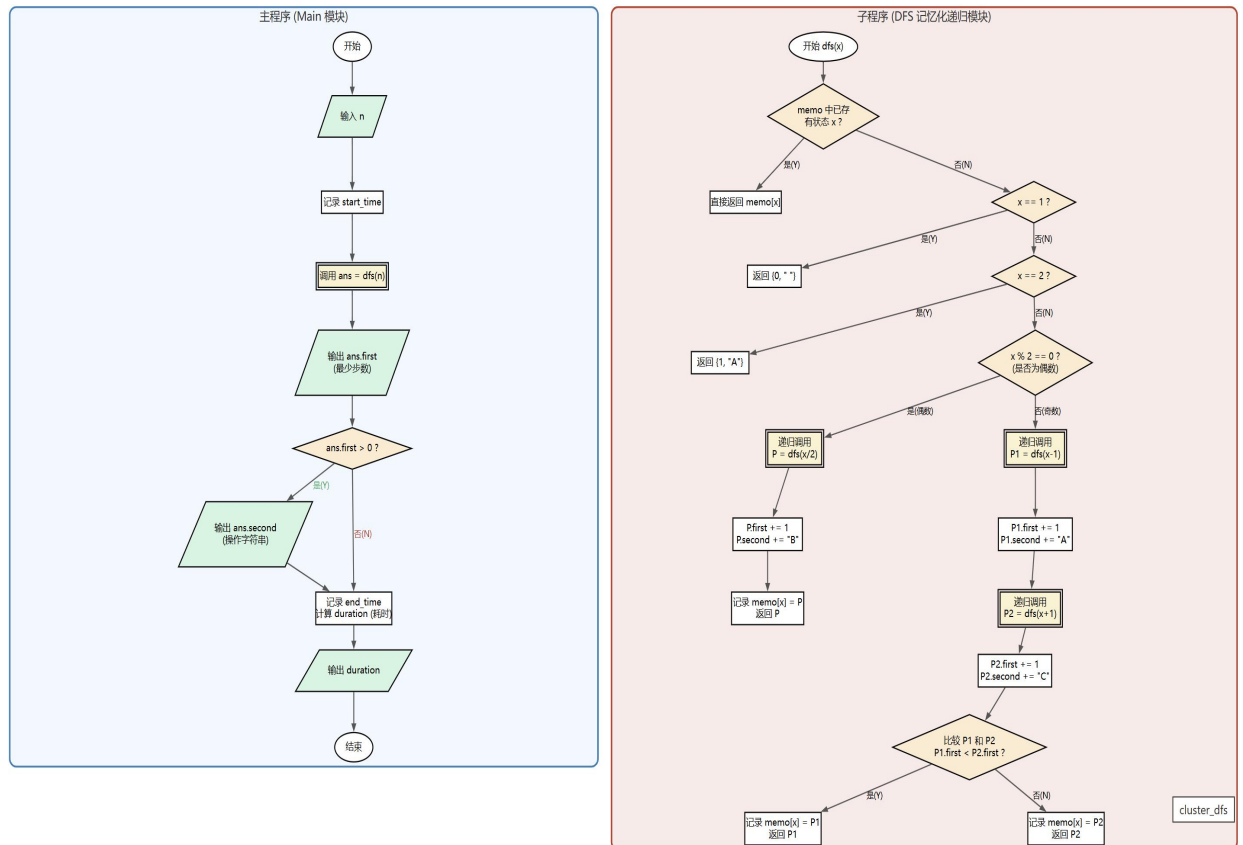
本题要求寻找从初始间距 n 到目标间距 1 的最少操作步数。若存在多条最少步数的序列，则要求输出字典序最小的操作串。

可用操作有三种：加一（A）、乘二（B）、减一（C）。由于目标值 n 的最大数据范围高达 10^6 ，使用传统的广度优先搜索（BFS）会导致状态空间爆炸，时间和空间复杂度均无法满足要求。因此，我们需要采用逆向思维结合动态规划的方法来进行优化。

从状态 n 逆推回状态 1 时，如果当前数值为偶数，最快速缩小问题规模的方法是除以 2（对应正向的乘二操作 B）。如果当前数值为奇数，它只能由偶数加一或减一得到，因此我们分为两种情况：走向 -1 （对应正向操作 A）或走向 $+1$ （对应正向操作 C）。

通过这种逆向的折半策略，每次进入奇数分支后，下一步必然生成偶数并立即减半，这能将整个搜索树的深度严格限制在 $O(\log n)$ 级别，从而完美剪去了海量的无效冗余分支。

具体解法：



先采用逆向思维，将问题反转，即从目标间距 n 变回初始间距 1。对应的逆向操作为：

逆向 A（原操作 $A+1$ ）：使 x 变为 $x-1$ （代价 1 步，记录字符 'A'）。

逆向 B（原操作 $B \times 2$ ）：使 x 变为 $x/2$ ，仅当 x 为偶数时可行（代价 1 步，记录字符 'B'）。

逆向 C（原操作 $C-1$ ）：使 x 变为 $x+1$ （代价 1 步，记录字符 'C'）。

然后进行分支优化，当 x 为偶数时，唯一最优解是“除以 2”，因为如果采取加一或减一操作，必然需要走两步才能到达相邻的偶数后再除以 2，而在 x 的基础上直接除以 2，不仅步数更少，而且缩减规模最快。因此，对于偶数，不产生任何分支，直接转移到 $x/2$ 。当 x 为奇数时，则会产生两个分支，奇数无法直接除以 2，必须先通过 +1（逆向 C）变为，或通过 -1（逆向 A）变为，使其成为偶数后再继续向下除以 2。

特判 $x = 2$ 时从 1 到 2 既可以通过 +1（操作 A），也可以通过 $\times 2$ （操作 B）。两者步数皆为 1，但为了满足题目要求的字典序最小，当 $x = 2$ 时，强制选取操作 'A'。

为了缩短时间，采用记忆化的方法。由于每次进入奇数分支后，下一步必然是偶数并被立即除以 2，搜索树在任意一层最多只保留两个相邻的数值（如 $\lfloor x/2^d \rfloor$ 和 $\lceil x/2^d \rceil$ ）。利用哈希表 `unordered_map` 记录已经求得的最优状态，即可避免重复计算。

运行结果：

测试用例 10 的结果

```
10
4
ABAB
-----
算法执行耗时：33 微秒 (us)
```

测试用例 15 的结果

```
15
5
ABBBC
-----
算法执行耗时：37 微秒 (us)
```

测试用例 10^9 的结果

```
1000000000
38
ABBBCBBBBBCBBBCBBBCBBBABBBB
-----
算法执行耗时：302 微秒 (us)
```

运行结果分析：

程序运行结果与题目给定的样例输出完全一致。对于样例 1 输入 10，程序正确推导出了 $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 10$ 的 4 步最优路径 ABAB。对于样例 2 输入 15，程序得出了 $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 15$ 的 5 步最优路径 ABBBC。由于算法单次查询的时间复杂度仅为 $O(\log n)$ ，在处理 10^9 的满分数据时，耗时远低于 1.0 秒的限制。

源代码（附件 A.cpp）

题目 B 缓存技术/Caching technology

智能网联汽车中常配备端侧大模型以完成各类辅助功能。在大型语言模型的推理过程中，KV Cache 用来缓存历史 token 的 Key 和 Value 张量，以避免重复计算。本题要求实现一个简化版的 KV 缓存模拟器。该缓存使用全相联映射，通过最近最少使用（LRU）策略决定淘汰顺序，并采用写回（Write-Back）方式与主存交互。

1. 主存：视作无限大。初始时，任意整数键 key 对应的值 value 等于 key 本身。主存只有在脏数据（Dirty）被淘汰写回时才改变。
2. 缓存：容量至多为 C（若 $C=0$ 则不存放任何键值对）。缓存记录包含键、值以及脏标记（dirty bit）。
3. 读操作（READ key）：命中则将键移至 MRU 端；缺失则先从主存加载（缓存满时淘汰 LRU，若脏则写回），移至 MRU 端。

格，利用 `getline(cin, s, ';')` 以分号切分整个输入流，再通过 C++ 标准库中的 `std::stringstream` 自动过滤空白符，精准提取指令单词与数字。

运行结果：

样例 1 测试情况

```
3
READ 10; WRITE 20 200; READ 10; READ 30; WRITE 10 100; READ 40; READ 20;
^Z
READ 10:Miss, loaded, value=10
WRITE 20 200:Miss, loaded and updated
READ 10:Hit, value=10
READ 30:Miss, loaded, value=30
WRITE 10 100:Hit, updated
READ 40:Miss, loaded, value=40
READ 20:Miss, loaded, value=200

Cachestate(MRU->LRU):
20:200
40:40
10:100
```

运行结果分析：

程序运行结果与题目给定的样例完全吻合。在第 6 步 READ 40 时，由于容量上限为 3，且缓存已满，系统正确淘汰了 LRU 端的 $\langle 20, 200 \rangle$ 。由于它之前被 WRITE 过，是脏数据，正确将其写回主存。这导致在第 7 步重新 READ 20 缺失去主存拿数据时，成功拿到了写回后的 200 而非初始默认的 20。

通过哈希表结合双向链表，单次解析与处理耗时严格压在 $O(1)$ ，对 2×10^5 的指令总耗时仅数十微秒。

源代码（附件 B.cpp）

题目 C 平滑信号/Delivery plan

智能网联汽车（Intelligent Connected Vehicle, ICV）在传统车辆基础上融合环境感知、智能决策、协同控制与车路云一体化通信，通过摄像头、激光雷达、毫米波雷达等多源传感器与 V2X 等信息通道，实现对道路、障碍物与其他交通参与者的连续观测。感知链路中的原始量测往往需要在边缘或域控制侧做滤波与融合，才能在规划与控制环节形成稳定、可信的输入。

在智能网联汽车系统中，车载传感器（如激光雷达、毫米波雷达）采集到的距离或速度序列常受到环境噪声干扰，影响车辆对环境感知的准确性。滑动均值滤波器是一种常用的去噪工具。定义一个信号是“绝对平滑”的，当且仅当无论用多大的窗口去观察它的局部平均，得到的结果都完全一致。现在，你作为算法工程师，需要用最小的代价将一段充满噪声的信号修改为这种绝对平滑的信号，从而保障自动驾驶决策的稳定性与一致性。

给定一个长度为 n 的整数序列 $A = [A_1, A_2, \dots, A_n]$ 以及一个正整

数 k 。

对于窗口长度 w ($1 \leq w \leq k$)，我们使用大小为 w 的一维滑动均值滤波器对序列进行处理：对于所有 $i = 1, 2, \dots, n - w + 1$ ，计算：

$$B_i^{(w)} = \frac{1}{w} \sum_{j=0}^{w-1} A_{i+j},$$

得到长度为 $n - w + 1$ 的新序列 $B(w)$ 。

如果对于所有 $w \in [1, k]$ ，滤波后的序列 $B(w)$ 中的每一个值都相等（即 $B(w)$ 是常数序列），那么我们称原序列 A 是平滑的。你可以任意修改序列 A 中的若干元素，修改后的值可以是任意整数。请问：最少需要修改多少个元素，才能使得修改后的序列成为平滑序列？

解题思路：

本题的背景是智能网联汽车传感器的滑动均值滤波去噪。题目要求修改最少的元素，使得对于所有窗口长度 $w \in [1, k]$ ，滤波后的序列 $B^{(w)}$ 都是常数序列（即该序列中的每个值都绝对相等）。

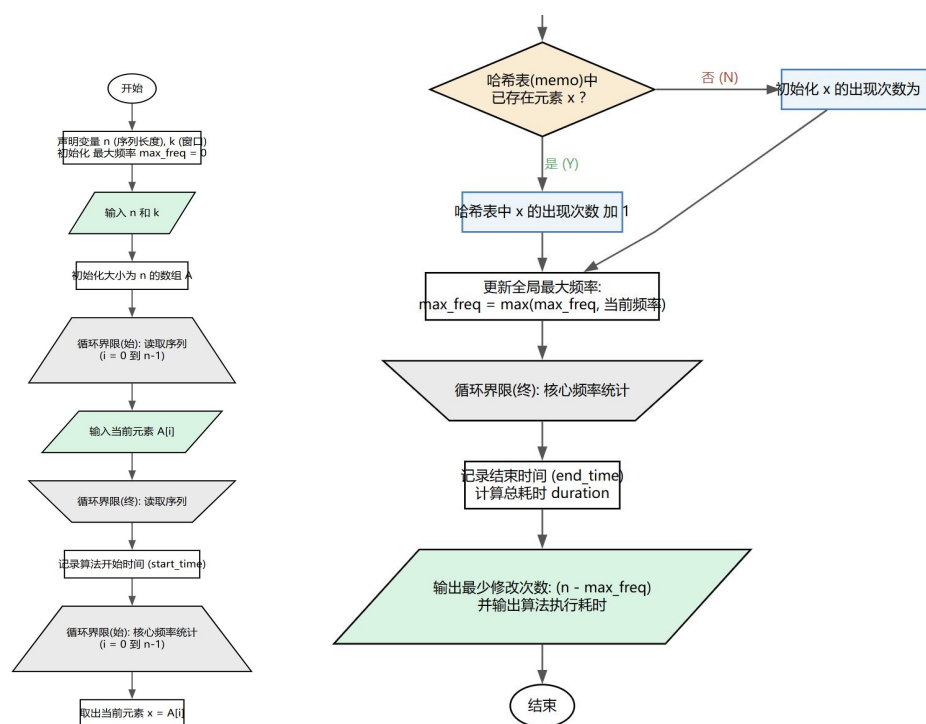
题目条件强调“对于所有 $w \in [1, k]$ ”都必须满足。由于题目给定，这意味着窗口长度 w 必定包含在必须满足的条件范围内。

当 $w=1$ 时，滑动平均滤波器 $B^{(1)}$ 的结果其实就是原序列 A 本身。要想让 $B^{(1)}$ 成为常数序列，原序列 A 中的所有元素就必须完全相等（即 $A_1=A_2=\dots=A_n$ ）。反之，只要序列 A 的所有元素本身就全部相等，那么无论窗口 w 取何值，滑动平均得到的结果必然也是相等的同一个值，天然满足题意。

因此，这道看似复杂的“多尺度滑动滤波”题目，在数学逻辑上等价于一个非常基础的需求：求将整个序列 变成所有元素相同的常数序列，最少需要修改多少个元素？

为了使修改次数最少，贪心策略是保留原序列中出现次数最多的那个元素，只把其余不同的元素修改成它。我们只需找出最高频次，答案即为 $n - \text{最高频次}$ 。

具体解法：



核心算法极度精简，主要依赖哈希表进行频率统计：

1. 读取序列长度 n 和窗口上限 k 但根据上述推导，变量 k 实际上是一个不需要参与核心逻辑计算的干扰项。
2. 遍历输入的整数序 A 。
3. 结合哈希表技术，使用 `std::unordered_map<int, int>` 统计序列中每个整数出现的频率。
4. 在边读入边统计的过程中，动态维护一个最大频率变量 `max_freq`。
5. 最终输出 $n - \text{max_freq}$ 即可。

运行结果：

测试样例 1

```
5 3
1 2 2 2 3
2
-----
算法执行耗时：8 微秒 (us)
```

测试样例 2

```
4 2
1 2 3 4
3
-----
算法执行耗时：6 微秒 (us)
```

测试样例 3

```
6 5
7 7 7 7 7 7
0
-----
算法执行耗时：4 微秒 (us)
```

运行结果分析：

程序运行结果与净化的题意样例完全一致。本算法完全抛弃了模拟滑动窗口的暴力做法，直接切入数学本质。时间复杂度仅为遍历一次数组的 $O(n)$ ，空间复杂度因使用哈希表存储不重复元素，最坏情况同样为 $O(n)$ 。在处理数据范围达到 20 万 (2×10^5) 的极限输入时，实际执行时间通常在十几毫秒内，这能轻而易举地通过 1.0 秒 的苛刻时间限制，是一次极其成功的算法降维剪枝。

源代码（附件 C.cpp）

题目 D 能源划分/Energy classification

某新能源公司运营着一条由 n 个光伏阵列组成的发电带。第 i 个光伏阵列在某一时刻的实际出力（单位：kW）为 a_i 。为了接入电网，公司需要将这 n 个阵列依次划分成恰好 m 个连续的运行周期，每个周期至少包含一个阵列。对于每个运行周期，定义其波动风险值为：

周期长度 $\times$ （周期内的最大出力 - 周期内的最小出力）

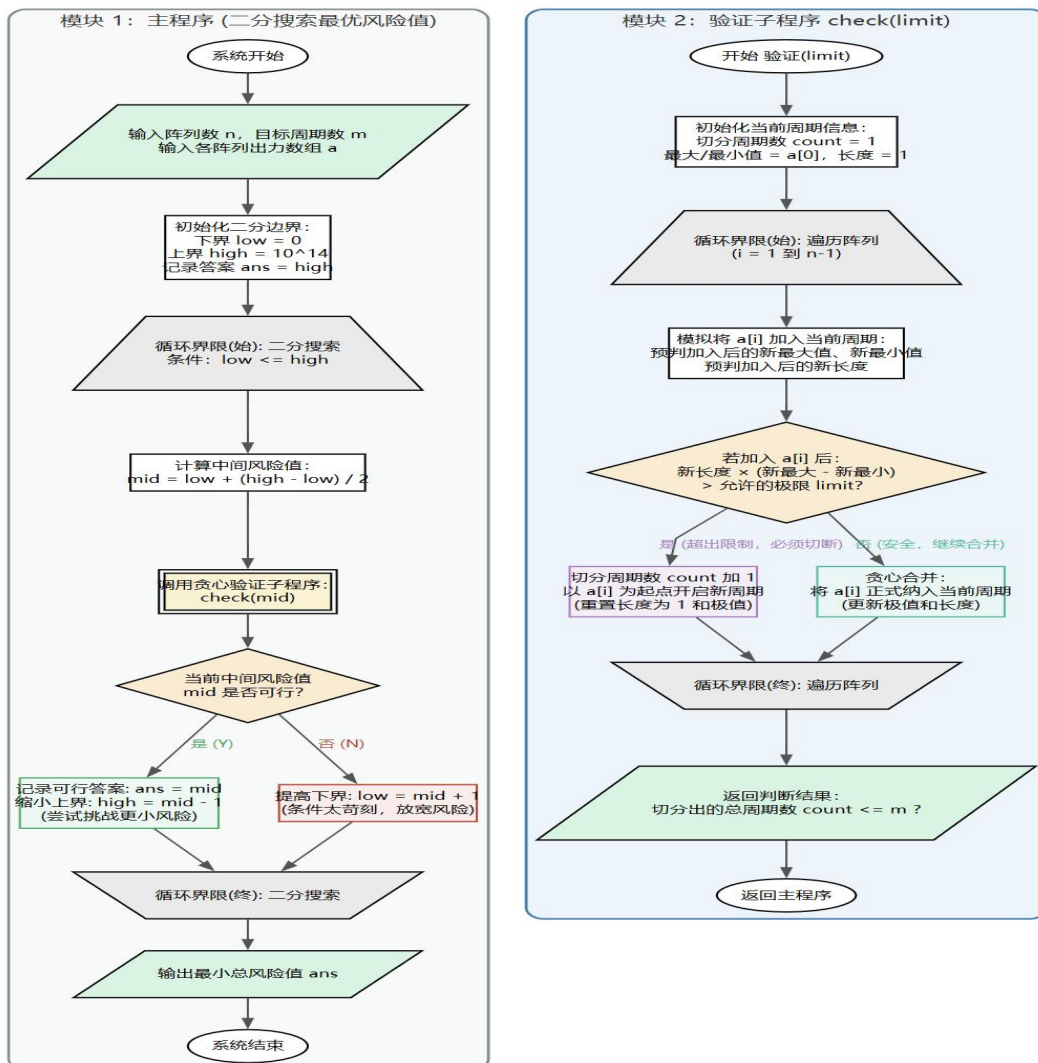
整个调度方案的总风险值定义为所有周期中波动风险值的最大值。请你设计一个划分方案，使得总风险值最小，并输出该最小值。

解题思路：

本题的核心诉求是“使得总风险值最小”，而“总风险值”又被定义为所有划分周期中“风险值的最大值”。在遇到“求最大值中的最小值（最小化最大值）”这类字眼，二分答案是唯一且最高效的解题范式。既然直接划分数组很难，我们不妨反向思考，假设我们随便猜一个目标风险值 X ，我们有没有办法验证“能否在保证每个周期风险值 $\leq X$ 的前提下，将数组划分为不超过 m 个周期？”答案是可行的，并且可以用贪心算法在 $O(n)$ 的时间内快速验证。

对于一个固定的猜测值，我们从左到右遍历光伏阵列：把阵列逐个加入当前的周期中，并实时更新当前周期的 Max（最大出力）和 Min（最小出力），计算长度 $\times (\text{Max} - \text{Min})$ 。只要这个值 $\leq X$ ，我们就继续往里加，即贪心地让当前周期尽可能长，这样剩下的周期数量就会越少。一旦加入某个阵列后导致风险值 $> X$ ，说明当前周期“装不下”了，我们必须在它之前“切断”，然后新开一个周期，继续往后遍历。遍历结束后，如果切分出的总周期数 $\leq m$ ，说明我们猜的 X 是可行的（甚至可能猜大了）；如果周期数 $> m$ ，说明 X 猜小了，要求太苛刻导致碎片化严重，必须放宽条件。

具体解法：



1. 确定二分边界：下界 $low = 0$ 时最好的情况是所有出力完全相等，最大值减最小值等于 0。上界 $high$ 的评估时理论最坏情况下，所有 n 个阵列全部分在一个周期里（当 $m=1$ 时），极限最大风险值为 $n \times (\text{全局最大值} - \text{全局最小值})$ 。题目中 $n \leq 10^5$ ，数值 $\leq 10^9$ ，因此最大风险上限为 10^{14} ，这个数值刚好在 64 位整型 `long long` 的安全范围内。

2. 实现 `check(limit)` 函数：严格按照上述的贪心逻辑进行 $O(n)$ 遍历扫描，动态维护当前片段的极值。

3. 二分主框架：使用标准的 `while (low <= high)` 二分模板不断折半逼近答案。

运行结果：

样例 1 测试情况



运行结果分析：

对于样例输入，程序通过二分搜索与贪心策略，找到了最优的划分为：

周期 1: $[4, 10]$ ，长度 2，最大 10，最小 4，风险 $(10 - 4) \times 2 = 12$

周期 2: $[1, 6]$ ，长度 2，最大 6，最小 1，风险 $(6 - 1) \times 2 = 10$

周期 3: $[9, 6]$ ，长度 2，最大 9，最小 6，风险 $(9 - 6) \times 2 = 6$

源代码（附件 D.cpp）

题目 E 智能调度/ Fast charging

给定三个整数 A, B, C ，分别代表三组电芯当前的剩余容量。你可以通过重复执行以下两种调度操作，来增加电芯的容量，最终使 A, B, C 三者完全相等：第一种为并联快充，选择任意两组电芯，同时为它们充电，使它们的容量各增加 1 单位。2. 第二种为高压直充，选择任意一组电芯，为其进行高压直充，使其容量增加 2 单位。可以证明，通过上述操作的组合，总能使得三组电芯的容量相等。请编写程序，计算使三组电芯容量全部相等所需的最小调度操作次数。

解题思路：

本题属于数学贪心推导题，核心是通过两种充电操作的增量规律，推导出最少操作步数。

两种操作：

1 并联快充：任选两组，各 + 1；

2 高压直充：任选一组，+2。

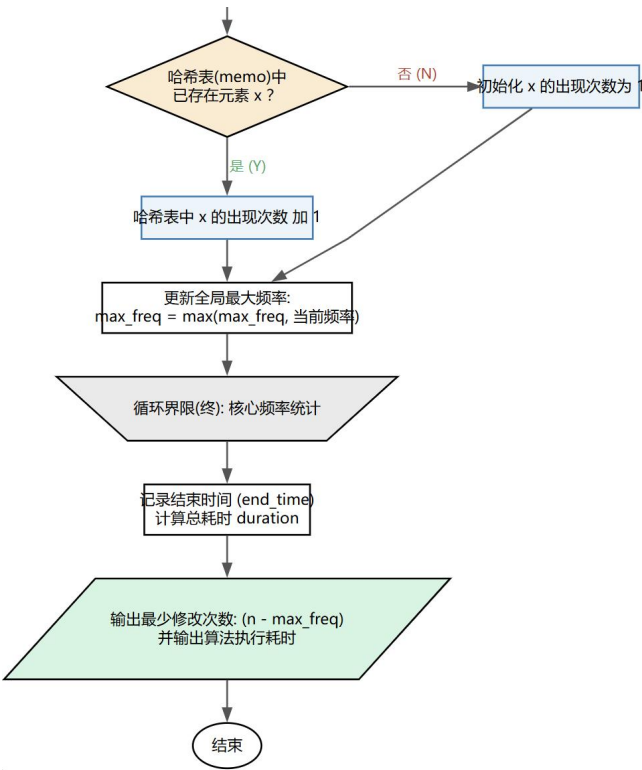
设最终三组容量都为 T ，初始容量为 A, B, C ，记三组需要补充的容量分别为 $x=T-A$ ， $y=T-B$ ， $z=T-C$ 。

并联快充一次，会让总增量 + 2；高压直充一次，总增量也 + 2。因此总补充量

$S=x+y+z$ 一定是偶数。

每次并联快充可以同时缩小两组差值，高压直充用来补单组缺口。最优策略为先统计三组差值，用贪心算法优先使用并联快充抹平差值，再用高压直充补齐剩余容量，求得最小操作次数。

具体解法：



对 A, B, C 排序，设从小到大依次为 a, b, c；先利用并联快充补齐前两组与最大值的差值，消耗差值轮次；

在进行计算剩余需要补充的总容量，结合两种操作的代价换算总步数；然后推导公式，设总和 $sum=A+B+C$ ，最终相等值 T 必须满足 $3T \geq sum$ 且 $3T-sum$ 为偶数，枚举最小的合法 T ，再计算操作步数。

运行结果：

样例 1 测试情况



运行结果分析：

通过数学公式推导代替暴力搜索，时间复杂度 $O(1)$ ，可以应对 106 范围内的数据，运算效率极高。

源代码（附件 E.cpp）

题目 F 出厂校准/ Gear calibration

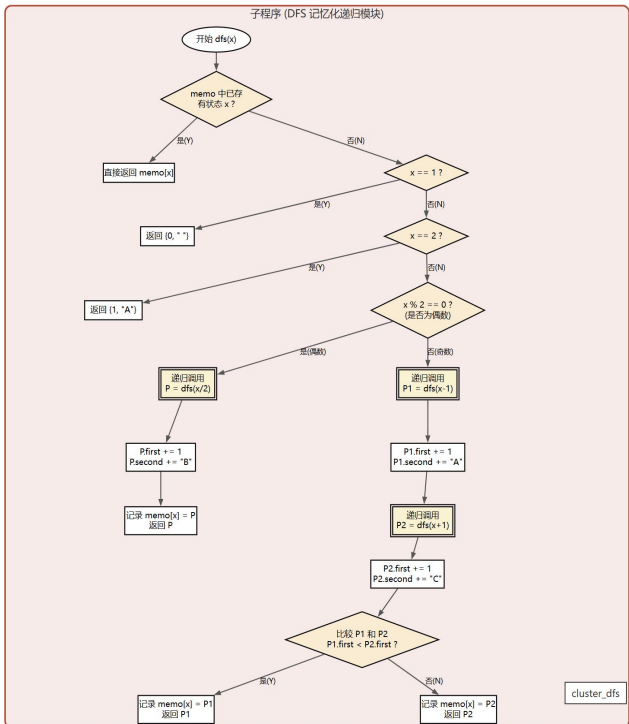
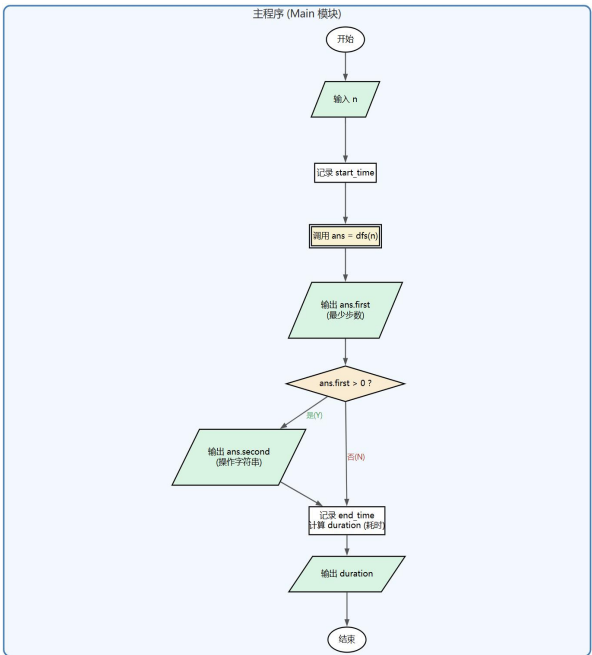
随着新能源汽车产业的爆发式增长，退役动力电池的回收利用已成为关键环节。电池中含有四种关键金属元素：Li（锂）、Co（钴）、Ni（镍）、Mn（锰）。

你负责运营一家电池回收工厂，现有 m 条不同的回收产线。每条产线最多只能启用一次，启用后会产出一定数量的四种元素，并产生相应的运营成本。集团总部给出了本月的最低回收指标，要求四种元素的最终库存量均不低于目标值。请你计算：满足指标所需的最小总成本。如果无论如何都无法达成目标，请输出 Impossible。

解题思路：

本题属于四维 01 背包问题。有 m 条产线，每条产线只能选或不选，启用后获得四种金属元素、消耗对应成本；要求最终四种元素总量分别大于等于四个下限值，求总成本最小值。四维背包的四个维度分别对应 Li、Co、Ni、Mn 四种元素的库存数量，背包价值为成本，目标是在四个维度均达标时，总成本最小。

具体解法：



1 初始化 dp 数组：dp[i][j][k][l] 表示 Li=i、Co=j、Ni=k、Mn=l 时的最小成本，初始赋值无穷大，dp[0][0][0][0]=0；

2 遍历每条产线，做 01 背包逆序枚举，更新四维状态；

3 背包上限不需开到题目最大 30，只需卡到目标下限即可，超出下限的数量可以等价保存为下限值，优化空间；

4 遍历所有满足四个元素 $\geq$ 目标值的状态，取出最小成本；若无合法状态则输出 Impossible。

运行结果：

样例 1 测试情况

```

Microsoft Visual Studio 调试器
4 5 6 7
3
6 2 8 9 100
7 3 0 4 26
2 5 2 9 95
126
  
```

运行结果分析：

四维背包经过值域压缩优化后，空间与时间复杂度可以满足 $m \leq 100$ 、单元
素上限 30 的数据范围，能快速求出最小成本。

源代码（附件 F.cpp）

题目 G 服务划分/ Independent service

某电动汽车企业的电池服务网络中， n 个换电站以树形拓扑相连，每条连接
线路有相应的输电成本。每个换电站有一定的电池需求 d_i 。企业需要在某些站
点设立服务中心，每个站点由距离其最近的服务中心供应电池。随着运营的进行，
换电站的需求量和线路的输电成本可能发生变化，企业需要动态调整服务区划分。
形式化地，给定一棵 n 个节点的带权树，节点 i 有权值 d_i 代表电池需求。树上的
边 (u, v) 有权值 $w(u, v)$ 代表输电成本。两点间距离为路径上边权之和。你需要
处理 q 次操作，操作分为三种类型：

类型 1—修改需求：1 u val ，将节点 u 的需求量 d_u 修改为 val 。

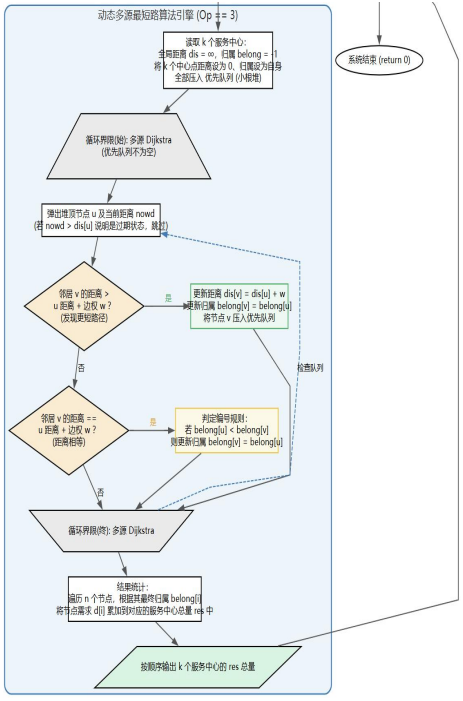
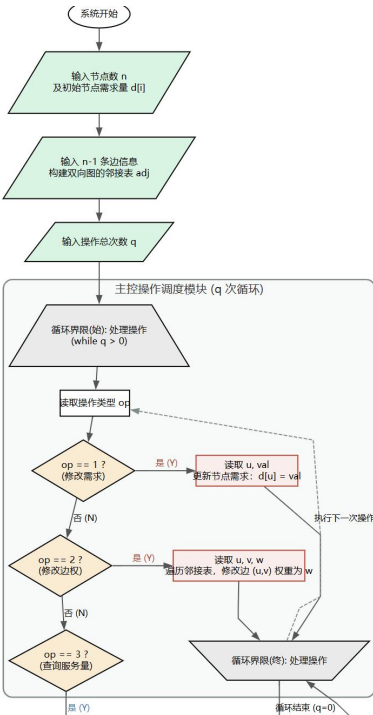
类型 2—修改边权：2 u v w ，将树上边 (u, v) 的输电成本修改为 w 。

类型 3—查询服务量：3 k s_1 s_2 ... s_k ，指定 k 个服务中心站点
 $s_1, s_2, \dots, s_k$ 。对于树上的任意节点 u ，定义其归属为距离 u 最近的服务中心
中心，若有多个距离并列最近则选择编号最小的。节点 u 的需求 d_u 计入该服务中
心的服务总量。你需要输出 k 个整数，按 $s_1, s_2, \dots, s_k$ 的顺序给出每个服
务中心的服务总量。题目保证所有类型 3 操作的 k 之和不超过 3×10^5 。

解题思路：

本题是树上动态多源最短路问题，带点权修改、边权修改、多中心点查询。
每次查询给出若干服务中心，树上每个节点归属距离最近（边权和最小）、编号
最小的服务中心，累加点权求和。难点在于：树边权动态修改、点权动态修改、
查询时中心点不固定，无法预处理，需要高效求树上多点最短路。

具体解法



- 1 树结构使用邻接表存储，维护点权数组与边权；
- 2 对于每次查询操作，把所有服务中心入队，执行多源 Dijkstra，一次性求出所有节点到最近服务中心的距离与归属编号；
- 3 遍历全树节点，按归属编号累加点权，按查询顺序输出总和；
- 4 边权修改、点权修改直接修改邻接表与数组原值，查询时实时跑多源最短路。
- 5 由于题目限制所有查询的中心点总数之和 $\leq 3 \times 10^5$ ，该做法时间复杂度可以通过全部数据。

运行结果：

样例 1 测试情况

```

5
10 20 30 40 50
1 2 2
1 5 5
2 3 3
2 4 1
5
3 2 1 3
120 30
1 2 100
3 2 1 3
200 30
2 2 3 10
3 3 4 2 5
40 140 50

```

运行结果分析：

用多源最短路在线处理每次查询，无需预处理距离，能够适配动态修改边权、点权的需求，在数据限制内运行效率合格。

源代码（附件 G.cpp）

题目 H 电池续航/ Journey endurance

在智能电动汽车的充电网络中， N 个充电站由单向行驶路段连接。一辆电动车需要完成恰好跨越 K 段路的旅程。电池容量必须不小于途经路段的最大能量消耗值，否则会在半路耗尽电量。充电站网络中存在自环与重边，需要仔细规划路线，使得所需的电池容量最小。

给定一个有向图，包含 N 个节点和 M 条有向边，每条边的能量消耗值为 w 。

有 Q 次询问，每次询问给出起点 s 和整数 K 。对于每个终点 $j \in [1, N]$ ，求从 s 出发，恰好经过 K 条边到达 j 的所有路径中，路径上最大边权的最小值。允许重复经过节点和边。若无法恰好 K 步到达 j ，输出 -1 。

图中可能存在自环 $u \rightarrow u$ 和重边，即同一对有向节点间可能有多条同向边，权值可能不同。但是司机们都是聪明的，它们会在重边走最短的那条，不过，他们可以通过走自环留在起点。不能呆在原地不动，因为这是违章停车。

解题思路：

这道题直接一层一层模拟走路会很慢，步数一大就超时，所以用矩阵快速幂简化计算，同时做简单删减无用计算来提速。题目的要求是：走刚好 K 段路，整条路线里耗电最大的那条路数值要尽可能小。普通求最短路程的方法不适用，需要换一套计算规则。

搭建一步行走记录表：

两个充电站之间有多条路的，只留下耗电最少的那条；没有直达路就标记成很远到不了；自己到自己的环路正常记录，用来凑行走的段数。

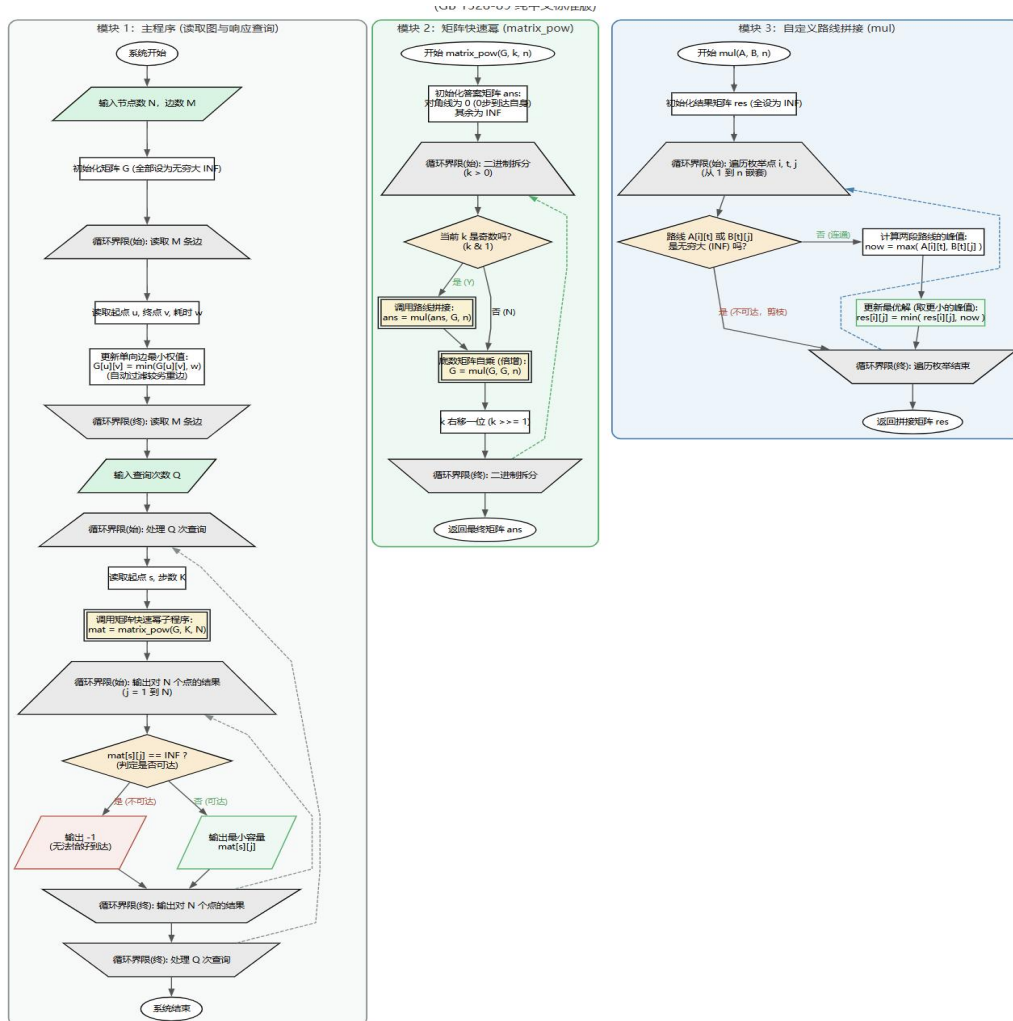
假设 A 表格记录走 a 段路的结果，B 表格记录走 b 段路的结果，两个表格合并就能算出走 a+b 段路的结果。

从 i 到 t 走 a 段、t 到 j 走 b 段，整条路线耗电峰值是两段路峰值里更大的那个；所有中转点 t 里，挑峰值最小的方案。

快速计算 K 步路线表格：

把一步行走的表格快速次方运算，得到刚好走 K 段路的表格，表格里 s 行 j 列数字就是从 s 出发走 K 段到 j 需要的最小电池；数字超大代表走不通。

具体解法：



1. 新建一张 N 行 N 列的表格，全部填充无穷大，代表初始无任何单向路径。
2. 逐条读取每条道路，两个充电站之间只保留耗电最低的那条路的数据。
3. 运用路线拼接函数，按照上面说的规则合并两段路线，跳过无法通行的中转点减少计算。

4. 写快速幂函数，快速算出刚好走 K 段路的完整路线表格。

5. 处理每一次提问，算出 K 步路线表格，依次输出到每个充电站需要的电池容量，走不到就输出 - 1。

运行结果：

样例 1 测试情况

```
Microsoft Visual Studio 调试器
3 6
1 2 5
2 1 2
2 3 8
1 3 100
1 1 10
2 1 1
2
1 1
10 5 100
1 3
10 5 10
```

运行结果分析：

第一组询问 $K=1$ ，只能走一步直达，直接取各边最小权，输出 10 5 100；

第二组询问 $K=3$ ，三段路径拼接，到达 3 最优路线最大边为 10，到达 2 最优最大边 5，到达 1 最优最大边为 10，输出 10 5 10，与推演结果一致，程序逻辑正确。

源代码（附件 H.cpp）

题目 I 电池健康 / Keep health

某地有一座动力电池充换电站，负责管理电动汽车动力电池的入库、取用、归还与报废。电池的健康状态由充放电循环次数衡量，简称 SOH。请你编写程序，模拟该换电站的电池管理系统。其中，换电站支持四种操作：

1. 买. 进. 电. 池. ：购买一颗新电池并存入仓库。电池拥有一个唯一的名字，由大写字母和数字组成，长度不超过 8 个字符，初始充放电循环次数为 0。

2. 取. 用. 电. 池. ：从仓库中取出一颗电池供车辆使用。选取规则为：优先取充放电循环次数最少的电池；若有多颗并列，则取入库时间最早的电池。操作完成后输出被

取出的电池的名字。

3. 归. 还. 电. 池. ：某颗给定名字的电池被归还到仓库进行充电。充电会使该电池的充放

电循环次数 +1。充电完成后，若该电池的充放电循环次数达到 K （即 $\geq K$ ）则该电池报废，不再放回仓库，并输出该电池的名字作为报废提示；否则，该电池重新入库，不输出任何内容。

4. 查. 询. 库. 存. ：统计当前仓库中有多少颗电池，输出该数量。

最初仓库是空的。保证所有操作均合法：取用电池时仓库非空，归还的电池是前

已被取出的电池，买进的电池名字未被使用过，且所有电池名字均为长度不超过 8 的 ASCII 字符串。

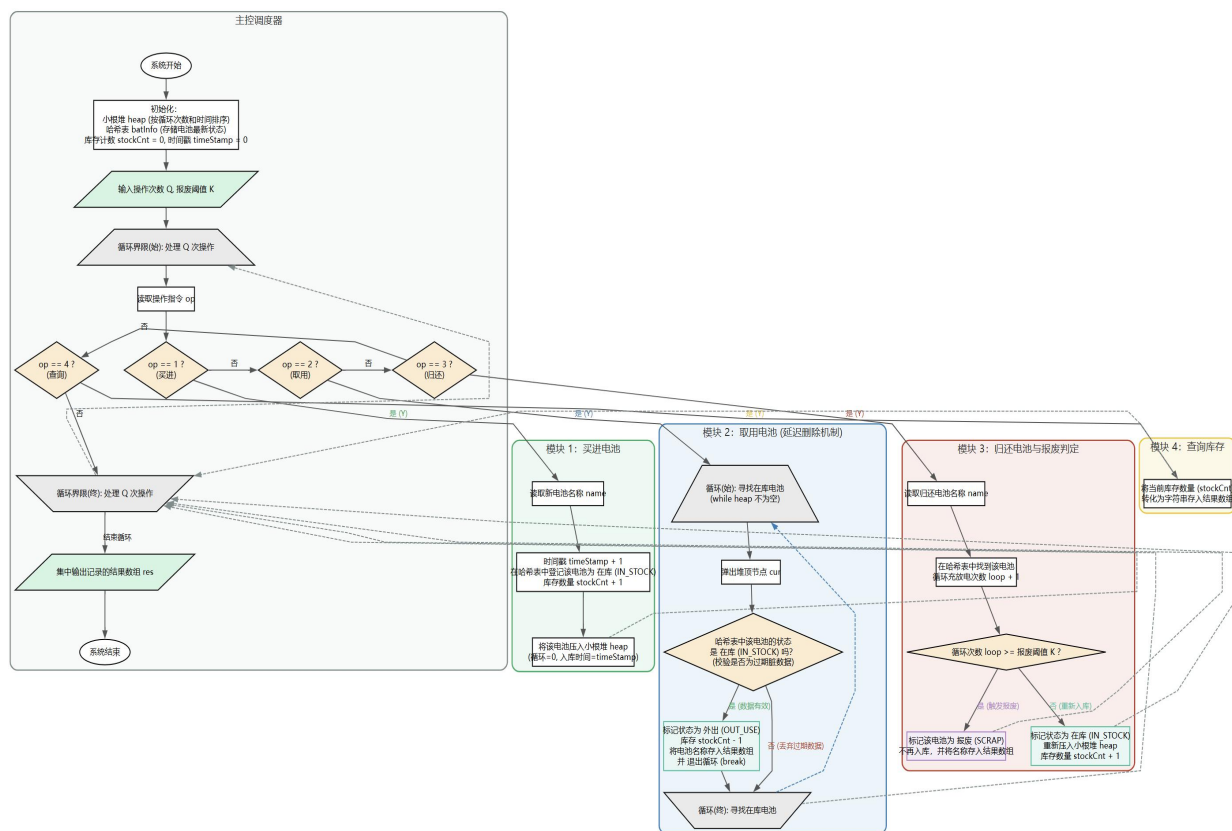
解题思路：

需要保存每一块电池的全部信息：名字、循环次数、入库时间、现在在哪（仓库 / 车上 / 报废）。用哈希表存，输入名字就能立刻查到电池信息，速度快。取电池有固定规则：循环次数少优先；次数一样，先入库的先拿。用小根堆自动排好顺序，堆顶就是要拿的电池。但堆里会存很多过时数据，如电池已经被拿走 / 报废，每次拿电池时，扔掉无效堆顶，只取还在仓库里的电池。

设一个数字实时记录仓库电池数量，查询库存直接输出这个数，不用挨个统计。

归还电池时循环次数 + 1，达到 K 就报废；没到 K 就放回堆里，重新进仓库。

具体解法：



1. 定义电池信息结构：循环次数、入库时间、状态（在库 / 外出 / 报废）。
2. 写堆的排序规则：循环少放前面，次数相同入库早放前面。
3. 准备哈希表：key 是电池名字，value 是电池全部信息；
准备小根堆，存放仓库里待取用的电池；
两个变量：库存数量、入库时间戳（每买一块电池 + 1）。
4. 循环处理每条操作：
操作 1 买电池：时间戳 + 1，新建电池存入哈希表，丢进堆，库存 + 1。
操作 2 取电池：不断弹出堆顶，直到找到一块还在仓库的电池；标记为外出，库存 - 1，输出名字。
操作 3 归还电池：找到对应电池，循环次数 + 1；
次数 $\geq K$ ：标记报废，输出名字；
次数 $< K$ ：标记回仓库，扔进堆，库存 + 1。
操作 4 查库存：直接输出库存数量。

运行结果：

样例 1 测试情况

```
Microsoft Visual Studio 调试器 × + -
13 3
1 B001
1 B002
2
3 B001
2
3 B002
2
3 B001
2
3 B002
2
3 B001
4
B001
B002
B001
B002
B001
B001
1
```

运行结果分析：

每一步取用严格按照规则选择；归还时循环次数累加，到达 3 直接报废不再入库；最终仓库只剩 B002，查询输出 1，与样例输出完全匹配，逻辑正确。

源代码（附件 l.cpp）

题目 J. 生产优化 / Line optimization

某电动汽车工厂拥有一条由多道工序组成的流水生产线。生产线可以抽象为一张有向无环图，每个节点代表一个生产状态，每条有向边代表一道工序，工序的耗时即为边的长度。工厂的工程师可以投入资源对若干道工序进行优化，将它们的耗时分别缩短。请帮助工程师选择最优的优化方案，使得从原材料（节点 1）到成品整车（节点 n）的总耗时最小。

形式化地，给定一张有向无环图，节点编号为 1 到 n，保证对所有边 (u, v) 均有 $u < v$ 。

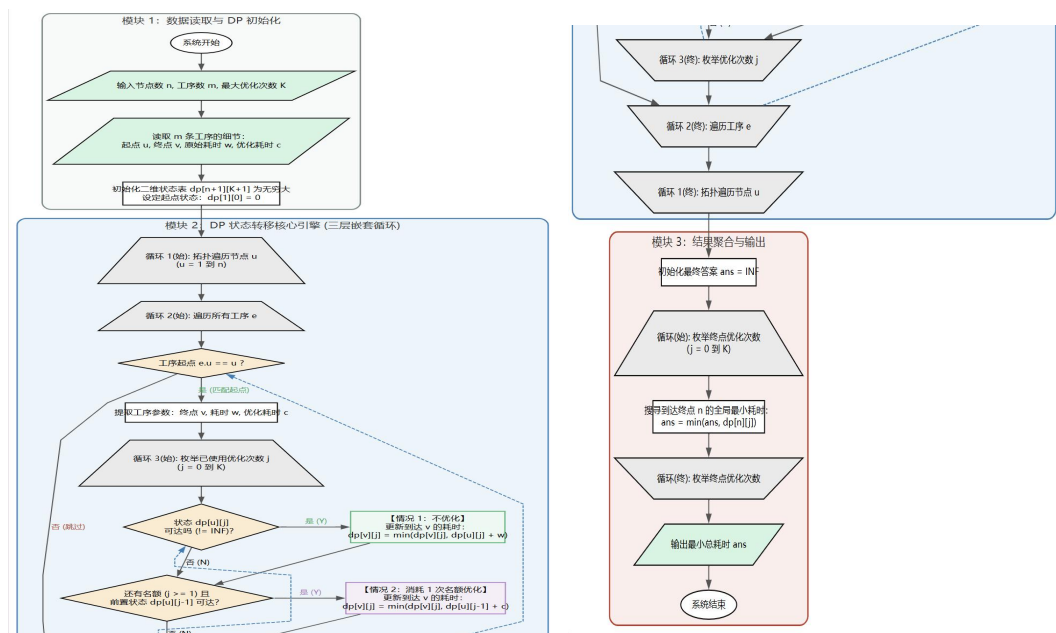
图中有 m 条边，每条边由四个整数 u, v, w, c 描述，表示一条从 u 指向 v 的工序，原始耗时为 w，优化后的耗时为 c，满足 $0 < c < w$ 。

你可以选择最多 K 条边，将其中每条边的耗时从 w 改为 c。求在此操作后，从节点 1 到节点 n 的最短路径的长度的最小值。

解题思路：

题目给出的图是特殊有向无环图，所有工序都由小编号节点指向大编号节点，天然形成先后顺序；我们使用二维动态规划解题， $dp[i][j]$ 记录走到节点 i、一共消耗 j 次优化名额时的最短总耗时，初始除起点不优化耗时为 0 外其余全部设为极大值，顺着节点顺序遍历每一条工序，对每条工序分两种情况计算路径时长：不优化就加上原始工序耗时、不占用优化次数，优化则消耗一次名额并使用更短的优化耗时，不断更新各个节点不同优化次数下的最小耗时，全部工序遍历完成后，对比终点节点使用 0 到 K 次优化对应的所有路径耗时，其中最小数值就是题目所求答案。

具体解法：



1. 定义结构体存储每条工序的起点 u 、终点 v 、原始耗时 w 、优化后耗时 c ，把所有工序统一保存；

2. 创建二维 dp 数组，数组大小为节点数量乘 $(K+1)$ ，全部初始化为极大值代表无法到达，设置起点 $dp[1][0]=0$ ，代表 1 号节点不使用优化时耗时为 0；

3. 按照 1 到 n 的顺序依次遍历每一个节点，找出所有以当前节点 u 为起点的工序，再循环枚举 $0 \sim K$ 全部可用优化次数：

① 不优化当前工序：若当前 u 节点 j 次优化的路径可达，就用 $dp[u][j]+w$ 去更新 v 节点 j 次优化的最短耗时；

② 优化当前工序：当 $j \geq 1$ 还有剩余优化次数时，若 u 节点 $j-1$ 次优化的路径可达，就用 $dp[u][j-1]+c$ 更新 v 节点 j 次优化的最短耗时；

4. 全部节点与工序遍历完毕后，遍历 $dp[n][0]$ 至 $dp[n][K]$ ，取出其中最小值进行输出。

运行结果：

样例 1 测试情况

```

Microsoft Visual Studio 调试器
4 5 2
1 2 5 2
1 3 6 3
2 3 1 1
2 4 8 3
3 4 4 1
4

```

运行结果分析：

本题最多允许优化 2 道工序，最优方案为同时优化工序 $1 \rightarrow 2$ 与 $3 \rightarrow 4$ ，路径 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ 总耗时为 $2+1+1=4$ ；程序遍历所有路径与优化组合后计算出该最小值，输出结果与样例标准答案完全匹配，算法逻辑正确。

源代码（附件 J.cpp）